

Semantic Analyzer

Kedar Sedai

Abstract—The semantic analyzer performs a two-pass walk over the AST: first generating function signatures, then type-checking each function body using stacked symbol tables for scopes. It validates variable declarations and uses, function calls, operator operand types, and control-flow conditions, reporting errors with source locations. It bridges syntactic analysis and later intermediate-code generation.

I. INTRODUCTION

Semantic analysis is phase 3. It runs after the AST exists and before IR/codegen. The parser only checks syntax(shape of the program). The semantic analyzer checks meaning:

- Is every variable declared?
- Do types match (int vs bool)?
- Are function calls correct (name, argument, count, argument types)?

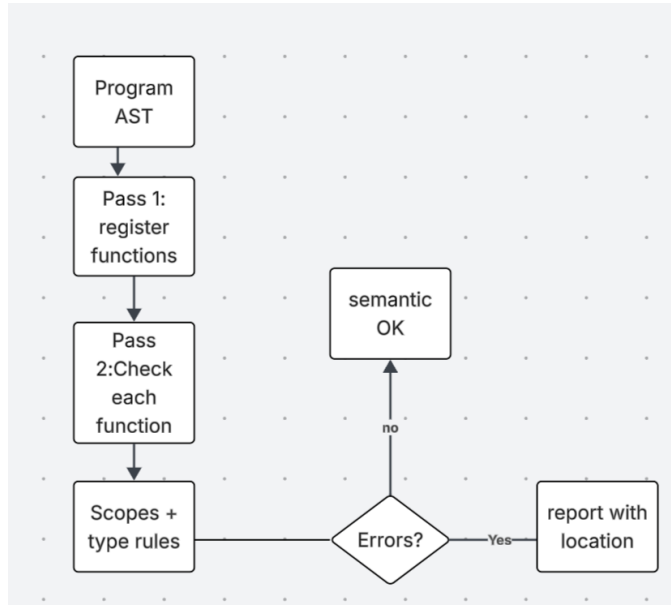


Fig. 1: Flowchart

II. DESIGN PROCESS

A. Define Semantic rules

The sample programs(factorial, hello), we listed rules that analyzer must enforce: The table below shows the Semantic rules for functions, variables, scopes, expressions, statements and void.

What did not implement yet:

- All paths must return (control- flow analysis)
- Implicit conversions (int -> bool)
- main must exist

keeping the first version small makes it easier to test and extend.

Category	Rules
Functions	No duplicate names; calls must match signature
Variables	Declare before use; no duplicate in same block
Scopes	Parameters + locals in nested blocks
Expressions	Each operator has allowed operand/result types
Statements	if /while need bool; return matches function type
void	void function: return; only, no value

TABLE I: Semantic rules

B. Choose analysis Strategy

Approach: AST visitor + symbol tables + type inference

Component	Purpose
Function symbol tree	Global map: name -> return type + parameter types
Scope stack	Stack of maps for variable (block nesting)
Type Checking	Recursive walk on Expr / stmt nodes
Two-pass analysis	Register all functions first, then check bodies

TABLE II: Analysis strategy

C. Design data Structures

Defined in *semantic_analyzer.hpp*

Function symbol(global):

```
struct FunctionSymbol {
    TypeKind return_type =
        TypeKind::Int;
    std::vector<TypeKind>
        parameter_types;
    SourceLocation location;
};
```

variable symbol(per scope)

```
struct VariableSymbol {
    TypeKind type = TypeKind::Int;
    SourceLocation location;
};
```

Storage:

```
const Program& program_;
std::unordered_map<std::string,
    FunctionSymbol> functions_;
std::vector<std::unordered_map<std::string,
    VariableSymbol>> scopes_;
```

- 1) functions_ - one global table.
- 2) scopes_ - stack: push on , pop on; function body gets its own scope

D. Pass 1: Register functions

```
void
SemanticAnalyzer::register_functions()
{
    for (const
        std::unique_ptr<FunctionDecl>&
        func : program_.functions) {
        // ...
        if (functions_.count(func->name)
            > 0) {
            error(func->location,
                "duplicate_function_" +
                func->name + "'");
            continue;
        }
        FunctionSymbol symbol;
        symbol.return_type =
            func->return_type;
        // ... collect parameter types ...
        functions_.emplace(func->name,
            std::move(symbol));
    }
}
```

For each FunctionDecl in the AST:

- Reject duplicate names
- Store return type and parameter type list
- Do not check the body yet

E. Pass 2: Check each function body

```
void
SemanticAnalyzer::check_function(const
FunctionDecl& func) {
    panic_mode_ = false;
    current_return_type_ =
        func.return_type;

    enter_scope();
    for (const Param& param :
        func.parameters) {
        declare_variable(param.name,
            param.type, param.location);
    }

    if (func.body) {
        check_block(*func.body);
    }

    exit_scope();
}
```

Process per function:

- Set current_return_type_
- Open scope
- Add parameters as variables (int n in factorial (int n))
- walk statements in the body
- close scope

F. Scope Management

Enter scope - push empty map:

```
void SemanticAnalyzer::enter_scope() {
    scopes_.emplace_back();
}
```

Declare variable - only in current scope (detects redeclaration in same block):

```
bool
SemanticAnalyzer::declare_variable(const
std::string& name, TypeKind type,
const SourceLocation& loc) {
    auto& scope = scopes_.back();
    if (scope.count(name) > 0) {
        error(loc, "variable_" + name +
            "'_already_declared_in_this_"
            "scope");
        return false;
    }
    scope.emplace(name,
        VariableSymbol{type, loc});
    return true;
}
```

Lookup variable - search from innermost to outermost(supports shadowing):

```
const
SemanticAnalyzer::VariableSymbol*
SemanticAnalyzer::lookup_variable(
const std::string& name) const {
    for (auto it = scopes_.rbegin(); it
        != scopes_.rend(); ++it) {
        auto found = it->find(name);
        if (found != it->end()) {
            return &found->second;
        }
    }
    return nullptr;
}
```

Blocks - each gets a new scope:

```
void
SemanticAnalyzer::check_block(const
BlockStmt& block) {
    enter_scope();
    for (const std::unique_ptr<Stmt>&
        stmt : block.statements) {
        if (stmt) {
            check_statement(*stmt);
        }
    }
    exit_scope();
}
```

Statement	What we check
varDecl	Evaluate initializer -> type must match declared type -> then declare_variable
if\while	condition must be bool -> check branches
return	void: no value; else: value required and type = current_return_type_
ExprStmt	Check expression types (e.g.) calls
Block	New scope; check each statement

TABLE III: Statement checking

G. Statement Checking

check_statement() dispatches on stmt:kind:
variable declaration(order matters: check type before adding to scope)

```
case Stmt::Kind::VarDecl: {
  const VarDeclStmt& decl = stmt.var_decl;
  const TypeKind init_type =
    check_expression(*decl.initializer);
  if (!had_error_ &&
      !types_equal(init_type,
                   decl.type)) {
    // ... error: cannot
    // initialize int with bool ...
  }
  declare_variable(decl.name,
                   decl.type, decl.location);
  break;
}
```

This catches `int x = true` in `semantic_bad_type.minilang`

H. Expression type Checking

check_expression() returns the type of an expression and reports errors. Pattern: recursive descent on the AST, mirroring the parser's expression structure.

Expr kind	Result type	checks
Variable	lookup in scopes	undefined -> error
Unary -	int	operand must be int
Unary !	bool	operand must be bool
+ - * /	int	both operands int
&&	bool	both bool

TABLE IV: Expression type checking

I. Error reporting

Same style as the parser: location + message on stderr.

```
void SemanticAnalyzer::error(const
  SourceLocation& loc, const
  std::string& message) {
  if (panic_mode_) {
    return;
  }
  panic_mode_ = true;
  had_error_ = true;
  std::cerr << "semantic_error_at_" <<
    loc.to_string() << ":_:" <<
    message << '\n';
}
```

- had_error sticky flag for exit code
- panic_mode_ - limit duplicate errors per function (reset in check_function)

J. Driver and tests

Driver semantic_main.cpp: full pipeline in one tool:

```
minilang::Lexer lexer(source,
  path);
minilang::Parser parser(lexer);
const minilang::Program program =
  parser.parse();

minilang::SemanticAnalyzer
  analyzer(program);
if (!analyzer.analyze()) {
  return 2;
}
```

Tests:

Program	Expected
factorial.minilang	OK
hello.minilang	OK
semantic_bad_undefined.minilang	fail: undefined variable
semantic_bad_type.minilang	fail int x = true

TABLE V: Analysis strategy

III. CONCLUSION

The Semantic Analyzer adds meaning to the AST produced by the parser. By combining global function symbols, nested variable scopes, and recursive type checking, it catches common programming errors before codegen, with lexical, syntactic and semantic phases complete, the compiler front end is ready for intermediate representation and LLVM lowering.